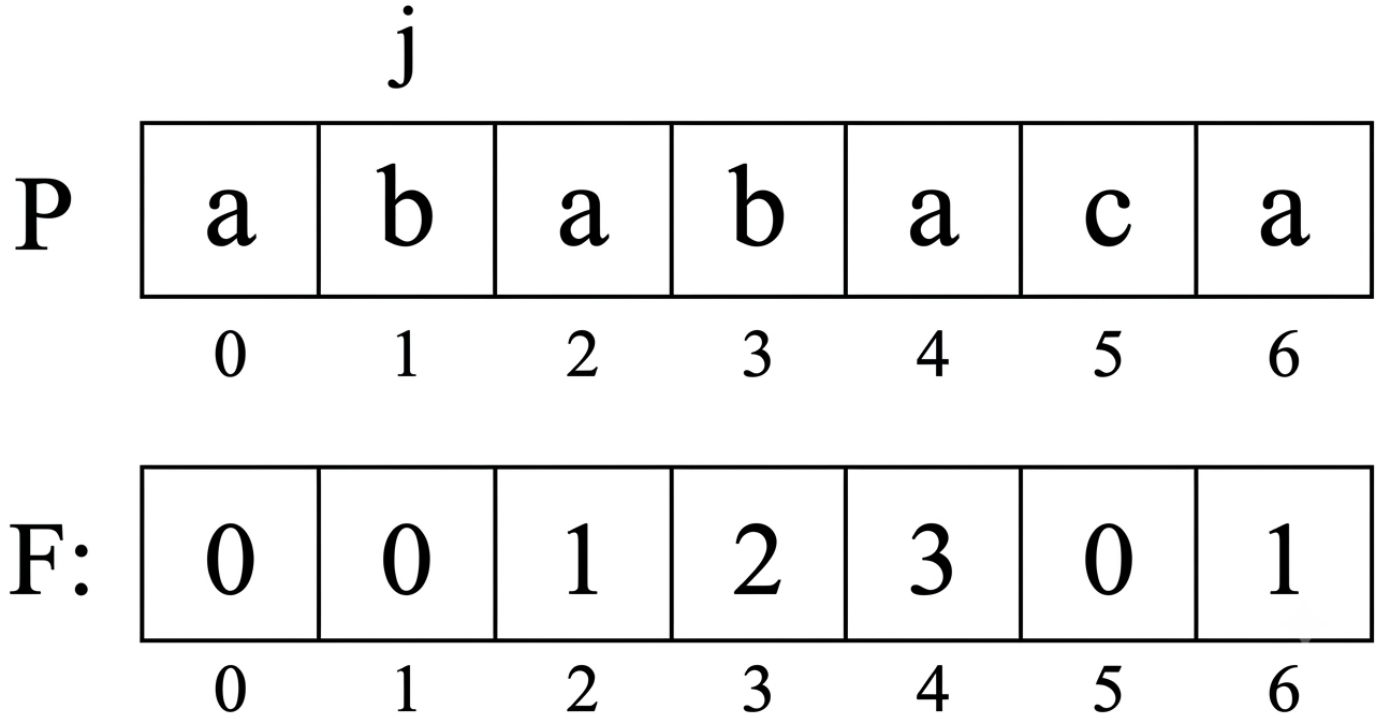
String Matching Algorithms

Dr. Om Kumar C.U.

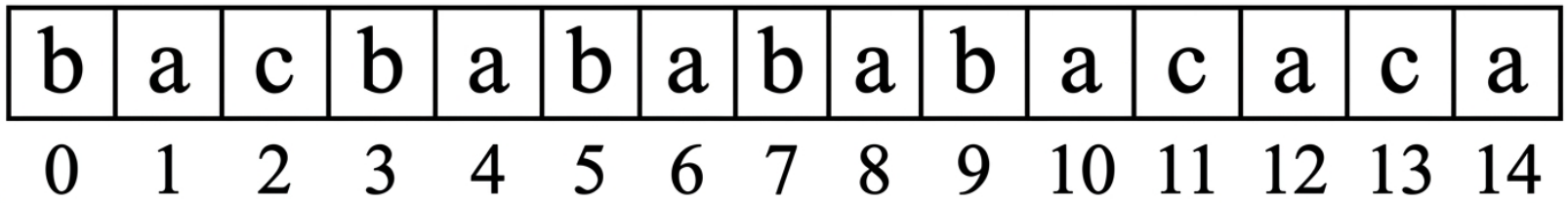
Pattern Failure Function

```
Algorithm Failrefuction (P)
{                                   // m
        F[0] ← 0
        i ← 1
        j ← 0
   while ( i < m)
   {    if ( P[i] == P[j])
          {   F[i] ← j+1
              i ← i+1
          }   j ← j+1
        else  if (j > 0)
                  j ← F[j-1]
        else
              F[i] ← 0
               j ← i+1
    }
```

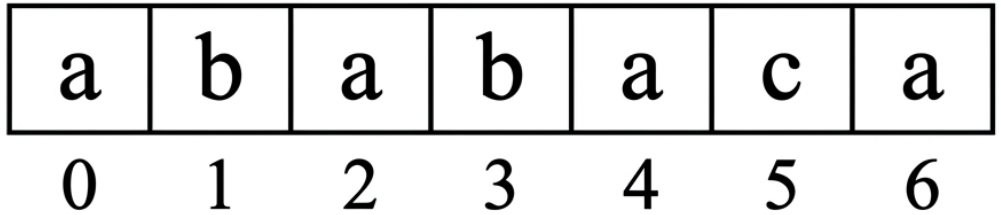


KMP

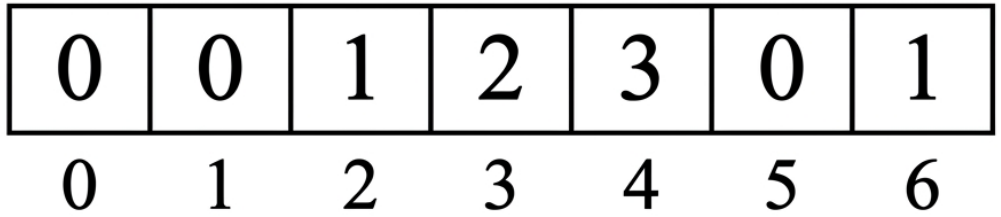
```
Algorithm KMP(t[0..n-1], P[0..m-1])
    F ← Failure(P)
    i ← 0, j ← 0
    while (i < n)
    {  if (t [i] == P[j])
       {  if (j == m-1) then
              return i - j
          else   i ← i+1,
    }            j ← j+1
    else {  if (j > 0) then
                 j ← F[j-1]
            else
          }          i ← i+1
}       return –1
```



P



F:



Rabin Karp Algorithm

	1	2	3	4	5	6
Text:	a	a	a	a	a	b

	1	2	3
pattern:	a	a	b

a — 1
b — 2
c — 3
d — 4
e — 5
f — 6
g — 7
h — 8
i — 9
j — 10

	1	2	3	4	5	6	7	8
Text:	a	b	c	d	a	b	c	e

n=8

	1	2	3
pattern:	b	c	c

m=3

a — 1
b — 2
c — 3
d — 4
e — 5
f — 6
g — 7
h — 8
i — 9
j — 10

Rabin Karp Algorithm

```
RABIN-KARP-MATCHER(T, P, d, q)
 1  n = T.length
 2  m = P.length
 3  h = d^(m−1) mod q
 4  p = 0
 5  t_0 = 0
 6  for i = 1 to m                          // preprocessing
 7      p = (dp + P[i]) mod q
 8      t_0 = (dt_0 + T[i]) mod q
 9  for s = 0 to n − m                      // matching
10      if p == t_s
11          if P[1..m] == T[s + 1..s + m]
12              print "Pattern occurs with shift" s
13      if s < n − m
14          t_{s+1} = (d(t_s − T[s + 1]h) + T[s + m + 1]) mod q
```

- Rabin–Karp compares the hash of the pattern with the hash of each text window; if the hashes match, it verifies the actual characters to handle collisions.

KMP	Rabin–Karp
Uses prefix function π	Uses hash values
Avoids unnecessary character comparisons using prefix information	Avoids unnecessary comparisons using hashing
q= number of matched characters	t_s= hash of current text window
Moves using $\pi[q]$after mismatch	Moves using rolling-hash formula
No hash collision issue	Hash collisions are possible, so verification is required

Example

What does Rabin–Karp do?

Suppose we want to find a pattern Pinside a text T.

For example: $T = 3141592653589793$ and $P = 9265$

Clearly, the pattern occurs here:

T = 3 1 4 1 5 9 2 6 5 3 5 8 9 7 9 3



The starting position is **6 in 1-based indexing**, or **shift** $s = 5$in the algorithm.

Parameters we need

Let's choose:

$T = 3141592653589793 \quad P = 9265$

Let $d = 10$

because we are dealing with decimal digits.

And choose a prime modulus: $q = 13$

So our input is:

T = 3141592653589793

P = 9265

d = 10

q = 13

```
RABIN-KARP-MATCHER(T, P, d, q)
 1  n = T.length
 2  m = P.length
 3  h = d^(m−1) mod q
 4  p = 0
 5  t_0 = 0
 6  for i = 1 to m                    // preprocessing
 7      p = (dp + P[i]) mod q
 8      t_0 = (dt_0 + T[i]) mod q
 9  for s = 0 to n − m                // matching
10      if p == t_s
11          if P[1..m] == T[s + 1..s + m]
12              print "Pattern occurs with shift" s
13      if s < n − m
14          t_{s+1} = (d(t_s − T[s + 1]h) + T[s + m + 1]) mod q
```

Variable	Meaning
n	Length of text
m	Length of pattern
d	Radix/base
q	Modulus
h	Highest-place multiplier
p	Hash value of pattern
t_s	Hash value of text window beginning at shift s

RABIN-KARP-MATCHER(T, P, d, q)

```
1   n = T.length
2   m = P.length
3   h = d^(m−1) mod q
4   p = 0
5   t_0 = 0
6   for i = 1 to m                    // preprocessing
7       p = (dp + P[i]) mod q
8       t_0 = (dt_0 + T[i]) mod q
9   for s = 0 to n − m                // matching
10      if p == t_s
11          if P[1..m] == T[s + 1..s + m]
12              print "Pattern occurs with shift" s
13      if s < n − m
14          t_{s+1} = (d(t_s − T[s + 1]h) + T[s + m + 1]) mod q
```

Calculate h

The algorithm says: $h = d^{m-1} \mod q$

Therefore:

$$h = 10^{4-1} \mod 13$$
$$= 10^{3} \mod 13$$
$$= 1000 \mod 13$$

Since: $1000 = 13(76) + 12$

we get: $\boxed{h = 12}$

Why do we need h?

Because it represents the contribution of the **leftmost digit** of a 4-digit window.

For example: 9265

↑ leftmost digit

What exactly are these hashes?

The pattern is: 9265

Its ordinary decimal value is:9265

The hash is: $9265 \mod 13 = 9$

So: $\boxed{p = 9}$

The first text window is: 3141

Its hash is: $3141 \mod 13 = 8$

So: $\boxed{t_0 = 8}$

Therefore:

Pattern hash = 9

First window hash = 8

Since they are different, the first window cannot be the pattern.

Shift s	Text window	Hash t_s	Pattern hash $p = 9$	Hash equal?	Actual match?
0	3141	8	9	No	No
1	1415	11	9	No	No
2	4159	12	9	No	No
3	1592	6	9	No	No
4	5926	11	9	No	No
5	**9265**	**9**	**9**	**Yes**	**Yes**
6	2653	1	9	No	No
7	6535	9	9	Yes	No
8	5358	2	9	No	No
9	3589	1	9	No	No
10	5897	8	9	No	No
11	8979	9	9	Yes	No
12	9793	4	9	No	No

Shift $s = 7$: collision
At: $s = 7$
the window is: 6535
Its hash: $t_7 = 9$
Pattern hash: $p = 9$
Therefore: p == ts is TRUE.
So the algorithm performs: P[1..m] == T[s+1..s+m]
Compare:
Pattern: 9 2 6 5
Window: 6 5 3 5

They are not equal.
Therefore **no match is reported**.
Then the algorithm calculates the next window's hash.

- The formula is: $t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \mod q$
- There are **four important pieces**:
- $$\boxed{d\left(\underbrace{t_s}_{\text{old hash}} - \underbrace{T[s+1]h}_{\text{remove first digit}}\right) + \underbrace{T[s+m+1]}_{\text{add new digit}}}$$

Current window: 3141

Next window: 1415